5장
같은 일을 여러 번 반복해야 할 때: 반복문

제어구조는 명령어를 실행하는 순서, 즉 프로그램의 흐름을 제어하는 구조입니다. 제어구조는 세 가지가 있습니다.

첫째, 순차구조는 위에서 아래의 순서대로 실행하는 규칙입니다. 프로그램을 순차구조로 간주하고 설계한 후에, 선택구조는 조건을 충족할 때와 충족하지 않을 때 실행할 명령어가 달라지는 구조로, 상황에 따른 조건문으로 표현합니다. 셋째, 반복구조는 특정 명령어를 반복하는 구조입니다.

이 장에서 저 번째 제어구조와 반복구조를 배워 보겠습니다.

▼ 표준 제어구조

↓	⅄	⅂

5.1

범위 내에서 반복하기: for

5.1.1 반복문이란

사람이 제한된 시간 안에 어떤 일을 매우 많이 반복하기는 시간적으로나 물리적으로 어렵습니다. 그래서 컴퓨터에 명령을 내려 짧은 시간 안에 수백, 수천 번 반복되는 일을 처리하게 합니다. 컴퓨터는 반복할 명령과 구간을 정해 주면 명령을 수백, 수천 번 반복할 수 있습니다. 이것이 제어구조 중 하나인 반복구조입니다. 파이썬에서는 반복구조를 다음과 같은 형식으로 작성하며, 이를 반복문이라고 표현합니다.

형식

```
for a in data:
    명령어 B
```

첫째 줄의 `for`에는 다양한 뜻이 있는데, 여기서는 '~하는 동안'으로 보면 됩니다. `in`은 '안에 있다, 속한다'는 의미이므로 첫째 줄은 '`a`가 `data` 안에 있는 동안'이 됩니다. `data`에는 반복할 구간이 들어갑니다. 구간은 어떤 지점과 다른 지점과의 사이이므로 시작과 끝을 나타내는 최소 2개 이상의 데이터가 있어야 합니다. 즉, `data`는 여러 개의 데이터가 있는 데이터 묶음입니다. `a`는 반복문에서 데이터 묶음 대신 사용할 변수로, `data`의 첫 번째 요소부터 마지막 요소까지 차례대로 변수 `a`에 저장되며 반복 구간을 설정합니다. 둘째 줄에 들여쓰기된 명령어 B는 `a`의 값이 변할 때마다 반복해서 실행됩니다. 따라서 반복문은 a가 데이터 묶음인 data 안에 있는 값일 동안 (반복해서) 명령어 B를 실행하라는 의미가 됩니다.



형식에 맞춰 원하는 데이터와 명령어를 넣으면 반복문이 실행됩니다. 아직 알쏭달쏭할 테니 바로 코드를 작성해 결과를 확인해 봅시다.

```
for a in [1, 2, 3, 4, 5]:
    print('안녕하세요.')
```

5개의 숫자가 대괄호(`[]`)로 둘러쌓여 있습니다. `data`에는 데이터 묶음이 들어간다고 했습니다. 그래서 이 코드에서는 1부터 5까지의 수를 대괄호로 묶어 `[1, 2, 3, 4, 5]`라는 데이터 묶음으로 만든 것이죠. 이 부분은 다음 장에서 상세히 배울 내용이라 여기서는 간단하게 '대괄호로 데이터 묶음을 표현한다'는 정도만 알면 됩니다. 이렇게 데이터 묶음이어야 `data` 위치에 넣을 수 있습니다. 그럼 한번 실행해 봅시다.

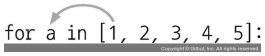
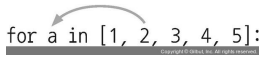
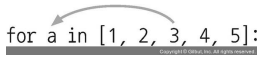
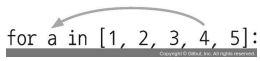
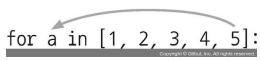
```
실행결과
안녕하세요.
안녕하세요.
안녕하세요.
안녕하세요.
안녕하세요.
```

`print('안녕하세요.')`는 1개인데 실행결과에는 '`안녕하세요.`'가 다섯 번 출력됩니다. 왜 그럴까요?

앞에서 `data`의 첫 번째 요소부터 마지막 요소까지 차례대로 `a`에 저장된다고 했습니다. 따라서 `[1, 2, 3, 4, 5]`의 데이터가 하나씩 변수 `a`에 저장될 때마다 반복문 내부에 있는 `print('안녕하세요.')` 명령어가 실행됩니다. 처음 반복문을 실행하면 첫 번째 요소인 `1`이 `a`에 들어가고 `print('안녕하세요.')` 명령어를 실행합니다. 데이터 묶음에는 여전히 데이터가 남아 있으므로 다시 `for` 문으로 돌아가 `a`에 두 번째 요소인 `2`가 들어가고 `print()`를 한 번 더 실행합니다. 같은 원리로 데이터 묶음의 마지막 요소인 `5`가 `a`에 들어갈 때까지 `print('안녕하세요.')`를 반복 실행합니다. `a`에 들어갈 데이터가 더 이상 없으면 반복을 멈추고 반복문을 종료합니다.

앞의 반복문에서 반복 구간은 `[1, 2, 3, 4, 5]`이고, 반복되는 명령어는 `print('안녕하세요.')`입니다. 반복문의 실행 과정을 정리하면 다음과 같습니다.

▼ 표 5-2 반복문의 실행과정

반복 회차	a에 저장된 값	반복문 내부
첫 번째	1	 a에 1이 저장되고, '안녕하세요.' 출력 `print('안녕하세요.')`
두 번째	2	 a에 2가 저장되고, '안녕하세요.' 출력 `print('안녕하세요.')`
세 번째	3	 a에 3이 저장되고, '안녕하세요.' 출력 `print('안녕하세요.')`
네 번째	4	 a에 4가 저장되고, '안녕하세요.' 출력 `print('안녕하세요.')`
다섯 번째	5	 a에 5가 저장되고, '안녕하세요.' 출력 마지막 데이터이므로 반복문 종료 `print('안녕하세요.')`

NOTE 반복문에서 오류가 발생하는 경우

반복문에서 오류가 발생하는 경우는 다음 두 가지로, 이는 조건문도 비슷합니다.

1. 콜론(:)을 입력하지 않았을 때 반복문에 콜론을 입력하지 않으면 `SyntaxError`가 발생합니다. 형식이 틀렸다는 의미죠. `SyntaxError`가 발생하면 가장 먼저 콜론(:)이 있는지 확인하세요.

2. 들여쓰기가 잘못됐을 때 반복문 내부에서 명령어를 입력할 때 앞에 4칸 들여쓰기를 하지 않으면 `IndentationError`가 발생합니다. 반복문에서도 콜론을 작성한 후 Enter를 누르면 자동으로 들여쓰기가 됩니다. 그런데 들여쓰기를 무시하거나 콜론을 입력하지 않은 채 Enter를 눌러 들여쓰기가 안 되면 오류가 발생합니다. `IndentationError`가 발생한다면 반복문의 둘째 줄에 들여쓰기가 제대로 됐는지 확인해 보세요!

반복문에서 변수 `a`의 값이 어떻게 변하는지 직접 확인해 보겠습니다.

```
for a in [1, 2, 3, 4, 5]:
    print(a, '을(를) 저장 중입니다.')
```

실행결과

```
1 을(를) 저장 중입니다.
2 을(를) 저장 중입니다.
3 을(를) 저장 중입니다.
4 을(를) 저장 중입니다.
5 을(를) 저장 중입니다.
```

반복할 때마다 `a`에 들어가는 값이 출력됩니다. 1부터 5까지 순서대로 출력되는 것을 보면 데이터 묶음에 있는 데이터가 변수 `a`에 차례대로 저장되는 것을 알 수 있습니다.

이 부분에서는 `a`가 변수고, 데이터 묶음의 첫 번째 요소부터 차례대로 변수 `a`에 저장됨을 이해하는 것이 중요합니다. `a`가 `data` 구간 안에 있는 값일 동안 명령어 B를 실행한다는 의미가 무엇인지 이제 이해될 겁니다.

만약 데이터 묶음의 값이 바뀌면 어떻게 될까요? 10개의 데이터 묶음으로 실행해 봅시다.

```
for a in [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]:
    print(a, '안녕하세요.')
```

```
실행결과
9 안녕하세요.
8 안녕하세요.
7 안녕하세요.
6 안녕하세요.
5 안녕하세요.
4 안녕하세요.
3 안녕하세요.
2 안녕하세요.
1 안녕하세요.
0 안녕하세요.
```

`print(a)`가 10번 실행되어 결과도 10번 나왔습니다. 그리고 `a`에 저장된 숫자가 데이터 묶음의 첫 번째 요소 9부터 마지막 요소 0까지 차례대로 변합니다. 이것이 바로 '`a`가 `data` 안에 있는 값일 동안'의 진짜 의미입니다. 데이터 묶음 안에 숫자를 다양하게 넣어서 테스트해 보세요.

반복문의 형식이 조금 익숙해졌을 거예요. 처음 봤을 때는 어려웠지만, 몇 번 실행해 보니 생각보다 쉽죠?

5.1.2 연속된 숫자를 range()로 간단하게 표현하기

반복문을 사용해 명령어를 5번, 10번 반복하는 프로그램을 작성했습니다. 그런데 이보다 더 많이 반복해야 하는 경우에는 어떻게 할까요? 200번을 반복한다면 1부터 200까지 숫자를 데이터 묶음 안에 넣어야 할까요?

이 문제를 간단하게 해결해 주는 `range()`라는 명령어가 있습니다. range는 우리말로 '범위'를 뜻합니다. 범위는 시작과 끝이 있습니다. 그래서 `range()` 명령어에도 범위의 시작과 끝을 넣어 `range(시작 숫자, 종료 숫자)` 형식으로 사용합니다. 다음 코드를 실행해 봅시다.

```
for a in range(1, 20):
    print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

```
실행결과
1 선생님, 다시는 청소를 빼먹지 않겠습니다.
2 선생님, 다시는 청소를 빼먹지 않겠습니다.
...
18 선생님, 다시는 청소를 빼먹지 않겠습니다.
19 선생님, 다시는 청소를 빼먹지 않겠습니다.
```

반성문을 19번 반복해서 출력합니다. 왜 20번이 아닌 19번만 출력할까요? `range(시작 숫자, 종료 숫자)`에서 종료 숫자는 명령어를 종료하고 반복문을 빠져나가는 숫자를 말합니다. 따라서 20이 되면 `range(1, 20)`이 종료되므로 실질적인 숫자 범위는 시작 숫자 1부터 19(종료 숫자 - 1)까지입니다. 이를 데이터 묶음으로 표현하면 `[1, 2, 3... 18, 19]`가 됩니다. 이처럼 20번을 반복하려고 데이터를 일일이 적는 것보다는 `range()`로 반복할 범위를 넣어 주는 것이 간편합니다.

반복문의 실행과정을 봅시다. 처음에 `a`에는 `range(1, 20)`의 시작 숫자인 1이 들어가고 `print('선생님, 다시는 청소를 빼먹지 않겠습니다.')`가 실행됩니다. 두 번째 반복에서 `a`에 2가 들어가고 `print()`를 실행하며, 세 번째 반복에서도 `a`에 3이 들어가고 `print()`를 한 번 더 실행합니다. 이 과정을 19가 될 때까지 반복합니다. 반복문의 `data` 위치에 `[1, 2, 3... 18, 19]`를 입력한 것과 결과가 같습니다.

▼ 표 5-3 반복문의 실행과정

반복 회차	a에 저장된 값	반복문 내부
첫 번째	1	 a에 1이 저장되고, `print()` 실행 `print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')`
두 번째	2	 a에 2가 저장되고, `print()` 실행 `print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')`
…		
열 여덟 번째	18	 a에 18이 저장되고, `print()` 실행 `print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')`
열아홉 번째	19	 a에 19가 저장되고 `print()` 실행 range(종료 숫자 - 1)이므로 반복문 종료 `print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')`

그럼 20번을 온전하게 출력하고 싶으면 코드를 어떻게 바꿀까요? 먼저 코드를 바꿔 보고 실행결과를 확인하세요.

```
for a in range(1, 21):
    print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

간단하죠? `range(1, 20)`을 `range(1, 21)`로 바꾸면 됩니다. 1부터 시작해 종료 숫자 21에서 1을 뺀 20까지 반복하므로 원하는 결과가 나옵니다. 다른 방법도 있습니다. 종료 숫자가 1 증가해도 되지만, 시작 숫자가 1 감소해도 20번 반복하겠죠? 출력 숫자도 동일하게 1부터 20이 나오게 하기 위해 출력 명령어에 `a` 대신 `a + 1`로 변경해 보겠습니다.

```
for a in range(0, 20):
    print(a + 1, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

```
실행결과
1 선생님, 다시는 청소를 빼먹지 않겠습니다.
2 선생님, 다시는 청소를 빼먹지 않겠습니다.
...
19 선생님, 다시는 청소를 빼먹지 않겠습니다.
20 선생님, 다시는 청소를 빼먹지 않겠습니다.
```

0부터 19까지 출력하니 총 출력 횟수는 20번입니다. `a`에 들어 가는 숫자도 0부터 19까지라, `a + 1`로 코드를 수정했더니 1부터 20까지 잘 출력됩니다.

마지막으로, `range(시작 숫자, 종료 숫자)`에서 시작 숫자가 0일 때는 시작 숫자 표기를 생략할 수 있습니다. 즉, `range(0, 20)`은 `range(20)`으로 간단히 써도 됩니다. 단, 시작 숫자를 생략하는 것은 0으로 시작할 때뿐이며 `range(1, 21)`처럼 0이 아닌 숫자로 시작할 경우에는 생략할 수 없습니다.

```
for a in range(20):
    print(a + 1, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

NOTE range()에 간격 추가하기

`range(시작 숫자, 종료 숫자)`에서 숫자가 1이 아니라 2나 3씩 증가하도록 간격을 설정할 수 있습니다. 방법은 간단합니다. `range(시작 숫자, 종료 숫자, 간격)`으로 기존 명령어에 간격을 추가하면 됩니다. 간격이 2라면 `range(1, 20, 2)`라고 작성합니다. 이는 `a`에 1부터 시작해 20까지 2씩 간격을 주고 값을 넣으라는 의미입니다. 실행해 봅시다.

```
for a in range(1, 20, 2):
    print(a)
```

```
실행결과
1
3
5
7
9
11
13
15
17
19
```

간격을 3으로 하고 싶으면 `range(1, 20, 3)`으로, 4로 하고 싶으면 `range(1, 20, 4)`로 작성하면 됩니다. 이때는 0부터 시작해도 시작 숫자를 생략할 수 없습니다. `range()`는 괄호 안에 든 요소가 2개면 자동으로 시작 숫자, 종료 숫자라고 생각하기 때문입니다.

지금까지 반복문의 형식과 `range()` 명령어를 배워 보았습니다. `for a in data`에서 `a`는 `data`의 요소를 차례대로 담는 변수라서 변수명이 반드시 a가 아니어도 됩니다. b나 c 또는 number 등 원하는 대로 지정할 수 있습니다. 일반적으로 프로그래밍할 때 반복문에서 사용하는 변수는 `i`입니다. 왜 i인지에 관해서는 여러 의견이 있습니다. 반복자를 의미하는 iterator의 약자라는 의견이 지배적이긴 하지만, 프로그래밍이 처음 만들어지는 시점에 고등 수학에서 사용하는 문자가 i여서 그대로 가지고 왔다고도 하고, index의 약자로 i를 사용한다는 의견도 있습니다.

이 책에서도 앞으로는 프로그래밍 관습에 따라 반복문에서 `a` 대신 `i`를 사용하겠습니다.

```
for i in range(20):
    print(i, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

`a`를 `i`로 바꿔도 반복문은 정상적으로 실행됩니다. 오류가 발생하거나 결과가 이상하다면 두 줄 모두 `a`를 `i`로 바꾸었는지 확인해 보세요. 하나라도 `i`로 바꾸지 않으면 오류가 발생하거나 숫자가 모두 0(또는 다른 숫자)으로 출력됩니다.

NOTE 왜 오류가 발생하거나 같은 숫자가 출력되나요?

1) 주피터 노트북이나 코랩에서 새로운 파일에 코드를 작성하거나 런타임을 새로 시작할 때(메뉴 → 런타임 → 런타임 다시 시작) 다음과 같이 작성하면 `NameError`가 발생합니다.

```
for a in range(20):
    print(i, '선생님, 다시는 청소를 빼먹지 않겠습니다.') # ①
# 또는
for i in range(20):
    print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.') # ②
```

`NameError`는 3장에서 공부했듯이 변수를 정의(변수를 만들어 초깃값을 저장)하지 않은 상태에서 변수를 사용했을 때 발생합니다. ①에서는 변수 `i`를, ②에서는 변수 `a`를 만든 적이 없는데 사용하고 있어서 '`a`(또는 `i`)에 값을 저장한 적이 없는데 어디에서 값을 불러오라는 거야!'라며 컴퓨터가 명령을 이해하지 못했다고 표시하는 것입니다.

2) 숫자가 0이나 다른 숫자 하나만 연속해서 출력되는 경우도 있습니다. 주피터 노트북이나 코랩을 사용하다 보면 코드를 복사한 후 붙여넣기를 해서 다음처럼 셀별로 실행하는 경우가 많습니다.



▲ 그림 5-1 코랩에서 코드의 셀별 실행

이렇게 실행하면 첫 번째 코드에서 변수 `a`에 0이 저장된 상태로 오류가 나서 종료됩니다. 변수 `i`가 정의되지 않았는데 반복문 내부에서 사용하기 때문입니다(반복문에서 정의한 변수는 `a`입니다). 두 번째 코드에서는 새로운 반복문을 작성해 변수 `i`를 만들었으나 반복문 내부의 `print()`에서는 `i`가 아닌 `a`를 사용합니다. 두 번째 코드에서 변수 `a`를 만들지 않았지만, 첫 번째 코드에서 만든 `a`에 0이 저장되어 있으므로 이를 사용해 오류가 나지 않고 출력 명령어를 반복해서 실행합니다.

데이터 묶음의 값을 차례대로 저장할 변수로 `i`를 사용한다면 반복문의 내부에서도 `i`를 사용해야 합니다. 이 부분을 꼭 확인하세요.

5.1.3 반복문으로 간단한 프로그램 만들기

지금까지 배운 반복문과 `range()` 명령어로 간단한 프로그램을 작성해 보겠습니다.

숫자 합 구하기

1부터 50까지 숫자의 합을 구해 봅시다. 먼저 반복할 부분과 구간을 찾아야죠. 1부터 50까지 숫자의 합에서 반복되는 부분은 뭘까요? 덧셈이죠. 명령어가 반복되는 구간은 어딜까요? 문제에 이미 나와 있듯이 1부터 50까지입니다. 먼저 1부터 50까지의 합을 저장할 변수가 하나 필요합니다. 이 변수의 초깃값에 1부터 50까지 계속 더하고 이 값을 다시 변수에 저장하므로 더해지는 숫자를 반복 구간으로, 덧셈을 반복 명령어로 만들면 됩니다. 지금까지 정리한 것을 바탕으로 코드를 작성해 봅시다.

```
# 1부터 50까지 숫자의 합을 구하는 프로그램
result = 0
for i in range(1, 51):
    result += i
print(result)
```

```
실행결과
1275
```

반복문을 시작하기 전에 먼저 결괏값을 저장할 변수 `result`를 만듭니다. 이때 `result`에는 초깃값으로 0을 넣습니다. 계산에 사용하는 변수라면 초깃값이 정말 중요합니다. `result`에 초깃값으로 1을 넣는다면 이미 1이 들어 있는 상태에서 1부터 50까지 더하므로 전체 계산에서 1이라는 오차가 생깁니다. 따라서 `result`에 초깃값으로 0을 넣어야 오차 없는 결괏값을 얻을 수 있습니다.

다음으로 반복 구간인 1부터 50까지의 수를 `range()` 명령어를 사용해 데이터 묶음 위치에 넣습니다. 이때 반복 범위는 `range(1, 51)`을 사용해야 함을 잊지 마세요! 반복할 때마다 변수 `i`를 `result`에 더합니다. `result += i`는 `result = result + i`를 줄여 쓴 표현입니다. 덧셈을 50번 반복하고 나면 최종 결괏값을 출력합니다. `print(result)`는 들여쓰기를 하지 않았으니 반복문 외부에 있습니다. 따라서 반복문이 종료되고 나면 실행됩니다.

`result` 변수를 반복문 내부에 만들면 어떻게 될까요?

```
for i in range(1, 51):
    반복되는 명령어
    result = 0
    result += i
print(result)
```

```
실행결과
50
```

결과는 50이 나옵니다. 왜 그럴까요? `result`에 초깃값 0을 저장하는 코드 `result = 0`도 반복되기 때문이죠. `result` 변수를 만드는 코드를 반복문 내부에 넣으면 첫 번째 반복에서 `result`에 0을 넣고, 다음 줄에서 `result`에 1을 더합니다. 두 번째 반복에서 `result`에 다시 0을 넣은 뒤에 2를 더합니다. 이런 식으로 50까지 반복해도 더한 결괏값 `result`에 0을 넣는 것도 반복되기 때문에 결과적으로 `result`에 0을 넣고 여기에 마지막 `i` 값을 더한 50이 출력됩니다. 이는 반복문을 처음 배울 때 많이 하는 실수입니다. 다음 코드 역시 흔히 하는 실수 중 하나입니다.

```
result = 0
for i in range(1, 51):
    반복되는 명령어
    result += i
    print(result)
```

```
실행결과
1
3
6
...
1176
1225
1275
```

반복문 안에 `print(result)`가 있어서 반복될 때마다 숫자를 더한 결괏값이 출력됩니다. 첫 번째 반복에서 `result`의 초깃값 0에 숫자 1을 더한 상태가 출력되고, 두 번째 반복에서 첫 번째 반복의 결괏값(`result`)에 2를 더한 3이 출력되고, 마지막에는 1부터 50까지 모두 더한 값인 1275가 출력됩니다. 의도한 바는 1부터 50까지 모두 더한 최종 결괏값만 확인하는 것이므로 덧셈 과정이 모두 출력되도록 반복문 안에 `print(result)`를 넣을 필요가 없습니다.

TIP 지금까지는 프로그램이 매우 간단해 중간 과정을 확인할 필요가 없습니다. 하지만 뒤에 나오는 데이터 분석 프로젝트를 진행할 때는 다소 복잡한 반복문을 사용하기 때문에 과정을 눈으로 확인해야 할 때가 종종 있습니다. 따라서 반복문 안에 `print()`를 넣어 값이 변하는 것을 직접 확인하는 방법도 알아 두면 좋습니다.

이번에는 101부터 150까지의 합을 구하고 싶습니다. 코드를 어떻게 수정하면 될까요? 먼저 직접 수정해 본 후에 책을 보기 바랍니다.

```
result = 0
for i in range(101, 151):
    result += i
print(result)
```

```
실행결과
6275
```

반복 범위를 정하는 `range()` 안의 숫자만 바꾸면 됩니다. `range(1, 51)`은 1부터 50까지의 숫자 50개를 차례대로 반복한다는 의미였으니 101부터 150까지의 합을 구하려면 숫자만 `range(101, 151)`로 변경하면 됩니다.

숫자 곱 구하기
앞의 프로그램을 참고해 1부터 50까지 곱(1 × 2... × 49 × 50)을 구해 봅시다. 책을 보지 않고 먼저 직접 작성해 보세요.

```
result = 1
for i in range(1, 51):
    result *= i
print(result)
```

```
실행결과
30414093201713378043612608166064768844377641568960512000000000000
```

덧셈 대신 곱셈 결과를 얻으려면 `+=`를 `*=`로 바꾸면 됩니다. 3장에서 배웠듯이 `result *= i`는 `result = result * i`를 줄여 쓴 표현입니다. 그런데 결과가 이렇게 긴 숫자가 아니라 0이 나올 수도 있습니다. 무엇이 문제일까요? 첫 번째 줄의 `result = 0`을 `result = 1`로 바꾸지 않아서 그렇습니다. `result`의 초깃값이 0이면 어떤 숫자를 곱해도 계속 0이 됩니다. 그래서 이때는 초깃값을 1로 해야 합니다. 덧셈에서 0이 결괏값에 영향을 미치지 않는 숫자라면 곱셈에서는 1이 결괏값에 영향을 미치지 않는 숫자입니다.

구구단 출력하기

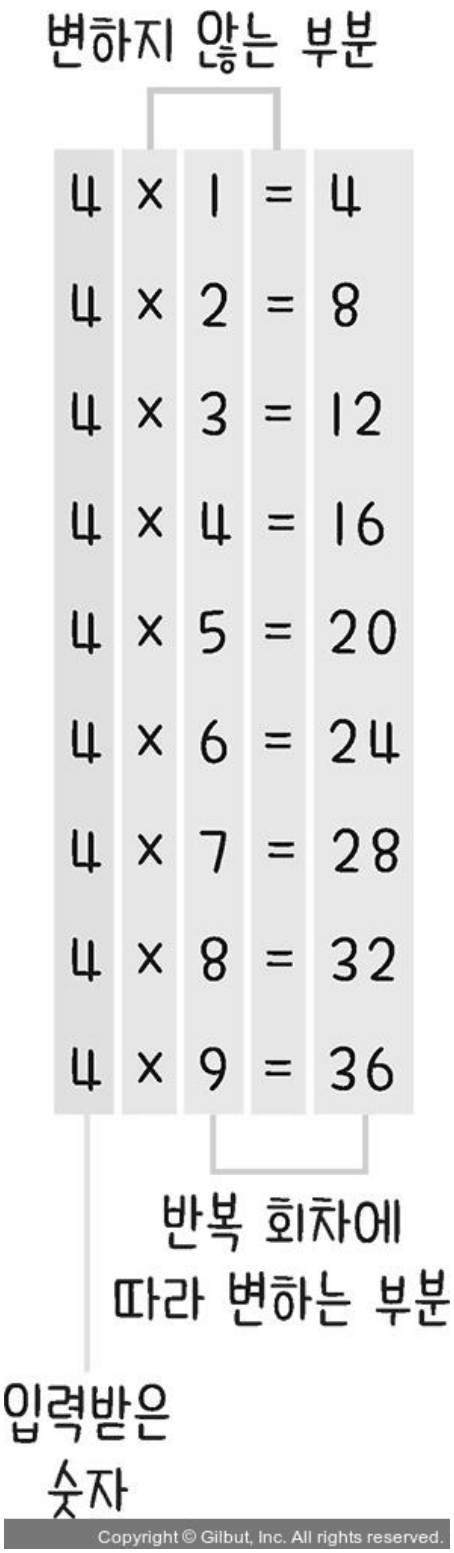
곱셈 코드를 배웠으니 숫자 하나를 입력받아 구구단을 출력하는 프로그램을 만들어 봅시다. 예를 들어, 4를 입력받으면 4단을 출력하면 됩니다. 구조를 생각해 봅시다. 일단 숫자를 입력받아야 합니다. 그리고 입력받은 값에 1부터 9까지 곱해서 결과로 '4 × 1 = 4, 4 × 2 = 8 ... 4 × 9 = 36'을 출력해야 합니다. 반복되는 부분은 어딘가요? 입력받은 숫자를 곱하는 부분으로, 4× ○ = ○ 형태로 표현할 수 있습니다. 반복 구간은 뭘까요? 1부터 9까지입니다.

구조를 고려해 반복문을 작성해 봅시다.

```
num = int(input('몇 단을 외울까요? '))
for i in range(1, 10):
    print(num, 'x', i, '=', num * i)
```

```
실행결과
몇 단을 외울까요? 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
```

먼저, 출력할 단을 `input()`으로 입력받습니다. 3장에서 배웠듯이 `input()`은 입력받는 값을 문자열로 취급하니 전체를 `int()`로 다시 감쌉니다. 앞에서 작성한 1부터 50까지 숫자 곱 구하기 코드를 참고하면 반복 범위는 `for i range(1, 10)`으로 작성할 수 있습니다. 그다음 줄에 반복문 내부에서 반복할 명령어를 들여쓰기로 입력합니다. 우리가 원하는 결과는 다음처럼 구구단을 출력하는 겁니다. 결과에서 변하지 않는 부분은 그대로 문자열로 출력하고, 변하는 부분은 변수를 활용합니다.



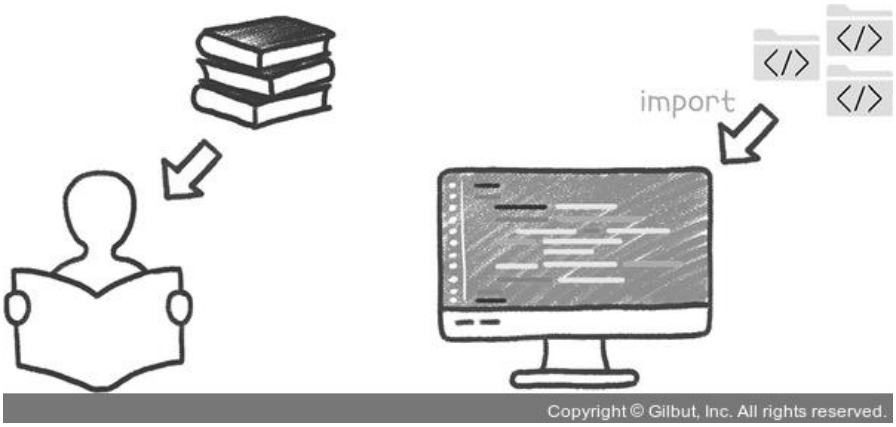
4는 입력받은 숫자가 저장된 `num` 변수입니다. `x`는 곱셈 기호인데, 변하지 않으므로 문자 그대로 사용합니다. 컴퓨터에서 곱셈 기호는 `*`이지만, 여기서는 보기 편하도록 알파벳 소문자 `x`를 사용했습니다. 그다음은 1부터 9까지 차례대로 변하는 변수입니다. 따라서 `for i in range(1, 10)`의 `i` 변수를 사용합니다. `=`도 변하지 않으므로 문자로 넣습니다. 마지막으로 반복 회차마다 변하는 곱셈 결과는 입력받은 숫자와 1부터 9까지 차례대로 곱하면 되므로 `num * i`로 작성합니다(실제 계산하는 부분이므로 곱셈 기호(`*`)를 사용하세요). 이를 모두 콤마(`,`)로 연결해 출력 명령어에 넣으면 `print(num, 'x', i, '=', num * i)`가 됩니다. 오류가 발생한다면 콤마를 제대로 입력했는지 확인해 보세요. 연결하는 요소가 많아서 콤마를 빠뜨리는 경우가 많습니다.

5.1.4 반복문으로 간단한 게임 만들기

무작위로 구구단 중 하나를 물어보면 정답을 말하는 '구구단을 외자' 게임이 있습니다. 이번에는 이 게임을 프로그램으로 만들어 보겠습니다. 구구단 게임은 숫자 2개가 있어야 하고 두 수를 곱해 답을 얻습니다. 따라서 무작위 숫자 2개를 얻는 과정이 필요합니다. 파이썬에는 무작위 수를 얻는 편리한 방법이 있습니다. 파이썬의 장점 중 하나인 라이브러리를 활용하는 방법입니다.

라이브러리에 속한 명령어 사용하기

라이브러리(library)란 우리말로 도서관을 뜻합니다. 도서관은 온갖 종류의 책, 문서, 기록 등의 자료를 모아 두고 찾아볼 수 있게 한 곳입니다. 프로그래밍에서 라이브러리도 비슷한 개념입니다. 특정 기능과 관련한 명령어를 사용하기 편하게 묶어 놓은 명령어 모음이라고 생각하면 됩니다.



▲ 그림 5-2 라이브러리

파이썬에는 다양한 기능의 라이브러리가 있습니다. 이 책 뒷부분에서는 그래프 시각화 라이브러리인 `matplotlib`(맷플롯립)을 사용해 데이터 분석 프로젝트를 진행합니다. 또한, 요즘 매우 주목받는 `keras`(케라스), `tensorflow`(텐서플로), `scikit-learn`(사이킷런) 등의 인공지능 관련 라이브러리도 있습니다.

파이썬에서 기본으로 제공하는 명령어도 있지만, 라이브러리에 속한 명령어를 사용해야 할 때도 많습니다. 라이브러리에 속한 명령어를 사용하려면 먼저 프로그램에 라이브러리를 추가해야 합니다. 라이브러리는 다음과 같이 추가합니다.

형식

```
import 라이브러리명
```

`import` 뒤에 추가할 라이브러리명을 넣습니다. `import`는 우리가 지금 작성하고 있는 파일에 라이브러리를 불러와 쓸 수 있게 하는 명령어입니다. 라이브러리를 추가하고 나면 라이브러리에 속한 명령어를 자유롭게 쓸 수 있습니다. 라이브러리에 포함된 명령어는 다음 형식으로 사용합니다.

형식

```
라이브러리명.명령어()
```

예를 들어, 구구단 게임에서 사용할 명령어는 `random`(랜덤) 라이브러리에 포함되어 있습니다. `random` 라이브러리의 `random()` 명령어는 다음처럼 사용합니다.

```
import
random random.random()
```

`random.random()`에서 앞의 `random`은 라이브러리고, 중간에 있는 도트(`.`)는 '안에 있는, 속한, 포함된'이라는 의미이며, 뒤의 `random()`은 사용할 명령어입니다. 따라서 '`random` 라이브러리에 속한 `random` 명령어를 사용하라'는 의미가 됩니다.



▲ 그림 5-3 라이브러리에 속한 명령어 사용하기

`random()`은 0 이상 1 미만의 숫자 중에서 아무 숫자나 하나를 뽑아서 돌려주는 명령어입니다. `random.random()`에서 어떤 값이 나오는지 `print()` 안에 넣어 출력해 봅시다.

```
import random
print(random.random())
```

```
실행결과
0.9880728327512837
```

소수점 아래로 긴 숫자가 보이네요. 무작위 숫자이기 때문에 여러분의 결과와 다를 수 있습니다. 여러 번 실행해 보면서 값이 어떻게 변하는지 확인해 보세요. 0과 1 사이의 다양한 실수가 나올 겁니다. 그런데 우리가 원하는 것은 이런 실수가 아니죠. 한 자릿수의 정수가 무작위로 나오길 바랍니다.

이럴 때 사용하는 명령어는 `random()`과 앞에서 배운 `range()`를 합친 `randrange(시작 숫자, 종료 숫자)`입니다. `randrange()` 명령어는 `range()` 명령어와 매우 비슷합니다. 시작 숫자와 종료 숫자가 있어 범위 내에서 무작위 숫자를 추출합니다. 또한, 종료 숫자는 결괏값에 포함되지 않습니다. 따라서 `random.randrange(1, 10)`을 하면 1부터 9까지의 숫자 중 하나를 무작위로 추출합니다. 무작위 수가 나오므로 실행결과는 생략합니다.

```
import random
print(random.randrange(1, 10))
```

또한, `range()`에서 간격을 설정할 수 있던 것처럼 `randrange()`도 간격을 설정할 수 있습니다. 간격을 2로 설정하고 여러 번 재실행해 보세요. 어떤 숫자가 나오나요?

```
import random
print(random.randrange(1, 10, 2))
```

`random.randrange(1, 10, 2)`는 1에서 9까지의 숫자 중 1부터 간격이 2인 숫자를 무작위로 뽑으라는 의미입니다. 1부터 간격이 2인 숫자는 1, 3, 5, 7, 9로 모두 홀수죠. 따라서 항상 홀수가 나옵니다. 간격을 다른 숫자로도 바꿔 테스트해 보세요.

`random.randrange()`로도 무작위 정수를 얻을 수 있지만, `random` 라이브러리에는 조금 더 편리한 `randint()`가 있습니다. `random()`과 `int()`가 합쳐진 `randint(시작 숫자, 종료 숫자)`는 시작 숫자와 종료 숫자 사이에서 무작위로 정수 하나를 돌려줍니다. 단, `randrange()`와 달리 종료 숫자도 포함합니다. 코드로 확인해 봅시다.

```
import random
print(random.randint(1, 10))
```

1부터 10 사이의 정수가 무작위로 하나씩 출력될 겁니다. 출력 명령어를 반복문 안에 넣어 여러 개의 무작위 값을 한 번에 확인해 보겠습니다.

```
import random
for i in range(20):
    print(random.randint(0, 10))
```

20번 반복하며 0부터 10 사이의 정수 20개가 출력될 겁니다. 여러 번 실행해 변하는 값을 확인하고 가장 작은 값인 0과 가장 큰 값인 10이 나오는지 확인해 보세요(무작위로 출력되므로 한 번 실행해서는 나오지 않을 수도 있습니다).

구구단을 외자(1)
라이브러리를 어떻게 사용하는지 알았으니 `random` 라이브러리를 사용해 구구단 게임을 작성해 보겠습니다. 처음이니 간단하게 다음처럼 결과를 출력해 봅시다.

```
실행결과
구구단을 외자! 구구단을 외자!
7 x 9 ? 63
3 x 7 ? 21
5 x 6 ? 30
2 x 7 ? 14
2 x 4 ? 8
6 x 9 ? 54
5 x 9 ? 45
8 x 8 ? 64
5 x 1 ? 5
3 x 3 ? 9
```

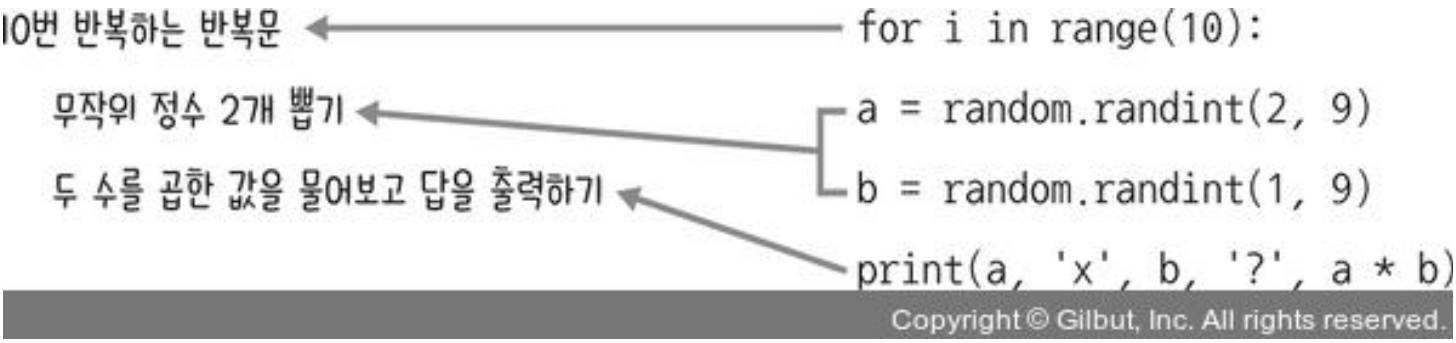
무작위 정수 2개를 곱하고 그 결과가 한 줄로 출력됩니다. 그리고 10번 반복됩니다. 반복되는 부분은 무엇일까요? '두 수를 곱해 결과를 한 줄로 출력하기'입니다. 그렇다면 반복 횟수가 10번이 되도록 반복문을 작성하고, 반복문 안에는 랜덤 구구단 문제와 결과를 한 줄로 출력하는 명령어를 넣어 주면 됩니다. 구조를 정리하면 다음과 같습니다.

```
10번 반복하는 반복문
    무작위 정수 2개 뽑기
    두 수를 곱한 값을 출력하기
```

10번 반복하므로 `for i in range(10)`으로 작성할 수 있습니다. 무작위 정수 2개가 필요하므로 `randint()`로 숫자를 뽑고 각각 다른 변수에 저장합니다. 첫 번째 무작위 정수를 저장하는 변수를 `a`, 두 번째 무작위 정수를 저장하는 변수를 `b`라고 합시다. `a`와 `b`에 넣을 무작위 정수는 어떻게 구할까요? 구구단은 2단부터 외우니 `a`에는 2부터 9 사이의 정수가 들어가야 합니다. 따라서 `random.randint(2, 9)`로 작성합니다. `b`는 그대로 1부터 9 사이의 정수가 들어가므로 `random.randint(1, 9)`로 작성합니다.

두 수를 곱한 값을 출력하는 부분은 실행결과 '7 x 9 ? 63'을 코드로 바꾸면 됩니다. 앞의 구구단 출력하기에서 어떻게 작성하는지 한번 해 봤죠? 변하는 부분과 변하지 않는 부분을 나눠서 생각하면 됩니다. 변하는 부분은 7, 9, 63입니다. 앞의 두 수는 변수 `a`, `b`의 값을 가져오고, 두 수를 곱한 값은 `a * b`로 구하면 됩니다. 변하지 않는 부분은 `x`, `?`이므로 문자로 넣어 줍니다. 모두 콤마로 연결해 `print()` 안에 순서대로 넣어 줍니다.

구조에 코드를 대입하면 다음과 같습니다(지면상 코드 부분은 들여쓰기를 생략합니다).



Copyright © Gilbut, Inc. All rights reserved.

그럼 나머지 코드도 작성하고 실행해 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a = random.randint(2, 9)
    b = random.randint(1, 9)
    print(a, 'x', b, '?', a * b)
```

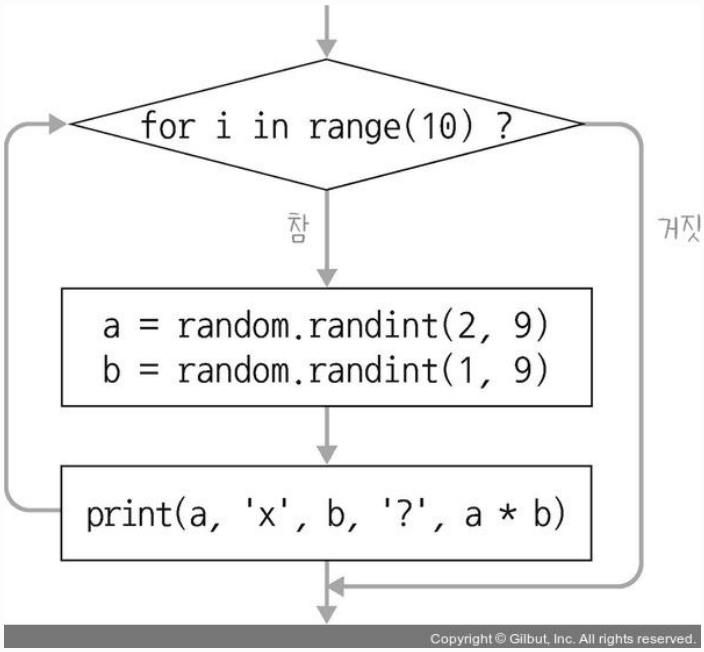
```
실행결과
구구단을 외자! 구구단을 외자!
8 x 7 ? 56
2 x 3 ? 6
8 x 6 ? 48
8 x 7 ? 56
7 x 9 ? 63
9 x 7 ? 63
2 x 5 ? 10
5 x 1 ? 5
7 x 1 ? 7
2 x 5 ? 10
```

정수가 무작위로 생성되므로 실행결과는 각자 다를 수 있습니다. 명령어를 사용하기 전에 라이브러리를 추가하는 것도 잊지 않았겠죠? 오류가 발생한다면 라이브러리를 추가했는지 확인해 보세요.

0부터 9까지 총 10번 반복되는 `for i in range(10)` 반복문을 만들고 반복문 내부에 들여쓰기 해서 반복되는 내용을 넣었습니다. 여러 번 프로그램을 재실행해 보면서 다양한 값이 나오는지 확인해 보세요.

NOTE 순서도로 표현한 반복문

이해하기 쉽게 설명하려고 프로그램의 구조를 순서도가 아닌 문장으로 표현했지만, 순서도의 반복문 표현도 알고 있으면 좋습니다. 다음은 앞의 프로그램을 표현한 순서도입니다.

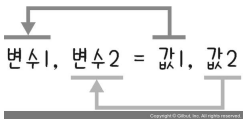


▲ 그림 5-4 순서도

순서도에서 반복문을 표현할 때 조건을 의미하는 마름모를 사용합니다. `for`의 의미를 조건문 형태로 표현한다면 '`data` 구간에 `i`에 저장할 요소가 남아 있는가'가 됩니다. `for` 반복문의 특성상 이 조건을 충족하면 자동으로 다음 요소를 `i`에 저장하므로 반복문은 조건과 실행할 명령어가 함께 구성되어 있다고 할 수 있습니다.

조건을 충족하면 반복문은 내부의 명령어를 실행합니다. 마지막 명령어 `print(a, 'x', b, '?', a * b)`가 실행되면 다시 반복 조건을 판단하는 부분으로 돌아가 `i`에 저장할 다음 요소가 있는지 확인합니다. 만약 있다면 조건에 대한 대답이 참이 되므로 반복문을 실행합니다. `i`에 현재 저장되어 있는 요소가 `data`의 마지막 요소라면 추가로 저장할 다음 요소가 없으므로 조건에 대한 답이 거짓이 됩니다. 반복 조건이 거짓이 되므로 반복문을 종료합니다.

코드를 보면 변수를 저장하는 두 줄이 있습니다. 지금까지는 `'변수 = 값'` 형식으로 하나의 변수에 값 하나를 넣었습니다. 그런데 여러 개의 변수를 한 줄로 정의할 수도 있습니다. 변수 여러 개에 값을 한 번에 넣을 때는 = 연산자 앞에 변수를, = 연산자 뒤에 값을 몰아넣고, 콤마로 각각 구분하면 됩니다. 그러면 첫 번째 변수에 첫 번째 값이, 두 번째 변수에 두 번째 값이 순서대로 저장됩니다.




```
a, b = 10, 20
print(a, b)
```

```
실행결과
10 20
```

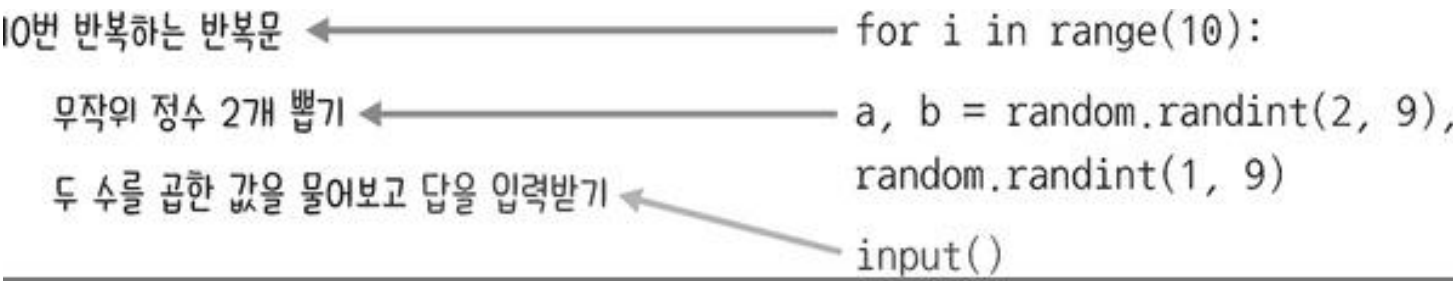
`a`에 10, `b`에 20이 차례대로 저장됩니다. 변수 여러 개에 한 번에 값을 저장하는 방법은 프로그램에서 자주 활용되니 꼭 기억해 두세요!

여러 변수에 값을 저장하는 방법을 앞의 코드에 적용하면 이전 코드와 동일하게 실행됩니다. 이렇게 하면 반복문 내부에서 `a`와 `b`에 랜덤한 값을 한 번에 저장할 수 있습니다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a, b = random.randint(2, 9), random.randint(1, 9)
    print(a, 'x', b, '?', a * b)
```

그런데 컴퓨터가 혼자 묻고 답하는 거라 재미가 없습니다. 컴퓨터가 문제를 내면 사람이 답을 입력하게 해서 진짜 게임처럼 만들어 봅시다.

컴퓨터가 답을 계산해 출력하는 것에서 답을 직접 입력받는 것으로 바뀝니다. 따라서 명령어도 `print()`에서 `input()`으로 바꾸고 답을 계산하는 부분도 없애야 합니다. 구조를 정리하면 다음과 같습니다(코드 부분 들여쓰기 생략).



Copyright © Gilbut, Inc. All rights reserved.

`print()`에서 `input()`으로 변경한 부분이 눈에 들어오죠. 그런데 `input()`으로 바뀌면서 구구단 문제를 물어보는 부분이 없어졌습니다. 이 부분을 해결해야 합니다.

이 부분은 `input()`의 입력 안내문에 구구단 문제를 넣으면 해결됩니다. `(a,'x', b, '?')`에서 `a`, `b`는 숫자형입니다. `input()`은 서로 다른 요소를 콤마로 연결할 수 없습니다(콤마로 연결해 출력하는 것은 `print()`에서만 가능합니다). 따라서 숫자형인 `a`, `b`를 `str()`으로 감싸 문자열로 변경한 후, 다른 문자와 `+`로 연결합니다. 그리고 그 문자열을 변수 `question`에 저장해 `input()`의 입력 안내문으로 넣는 방법을 사용하겠습니다. 그럼 정리한 대로 코드를 작성해서 실행해 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a, b = random.randint(2, 9), random.randint(1, 9)
    question = str(a) + 'x' + str(b) + '? '
    input(question)
```

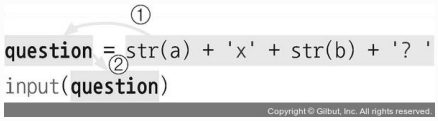
실행결과

```
구구단을 외자! 구구단을 외자!
7x9? 63
3x7? 21
5x6? 30
2x7? 14
2x4? 8
6x9? 54
5x9? 45
8x8? 64
5x1? 5
3x3? 9
```

컴퓨터가 무작위로 구구단 문제를 출제하고 사람이 답을 입력하길 기다리는 프로그램이 됐습니다.

NOTE 변수 줄이기

이 책에서는 코드를 이해하기 쉽게 단계를 빠짐없이 설명하고 있습니다. 그래서 구구단 문제를 저장하는 `question` 변수를 만들었지만, 사실 그러지 않아도 됩니다. 다음 두 줄만 봅시다.



첫째 줄에서 계산한 값이 `question` 변수에 저장됩니다. 둘째 줄에서 `question`의 값을 `input()`에 넣어 안내문으로 출력합니다. 따라서 첫째 줄의 `question`과 둘째 줄의 `question`은 같은 값이죠. 그렇다면 둘째 줄의 `question` 대신 `str(a) + 'x' + str(b) + '? '`를 넣어도 결과는 같으므로 두 줄을 다음처럼 합쳐도 됩니다.

```
input(str(a) + 'x' + str(b) + '? ')
```

이 형태는 3장에서 변수를 배울 때부터 봐와서 익숙할 겁니다. 이처럼 관계를 잘 따지면 변수 `question`을 만들지 않고도 코드를 작성할 수 있습니다. 두 줄이던 코드를 한 줄로 줄일 수 있으니 더 효율적이죠. 하지만 바로 작성하기 어렵다면 `question`을 사용해 단계별로 작성해도 됩니다. 반대로 변수를 사용하지 않는 것이 편하면 한 줄로 작성해도 되고요.

그런데 이 프로그램에는 문제가 있습니다. 프로그램을 재실행해 입력칸에 정답이 아닌 다른 값을 넣어 보세요. 정답이 아닌데도 아무 문제없이 다음 문제로 넘어갑니다. 다음 절에서 이 문제를 해결해 봅시다.

1분 퀴즈

1 다음 중 틀린 것을 고르세요.

```
result = 0
for i in range(10, 20):
    result += i ** 2
    (가)
(나)
```

① 이 코드는 `result`의 값을 반복해서 계산한다.

② 이 코드는 `range(10, 20)` 범위에 있는 숫자를 제곱한 후 모두 더한 값을 구한다.

③ `result += i ** 2`는 `i`가 10일 때부터 20일 때까지 총 11번 실행된다.

④ `result` 변수의 최종 값을 마지막에 한 줄로 출력하고 싶으면 (나)에 `print(result)`를 넣는다.

⑤ `result` 변수의 값이 반복문 안에서 변하는 과정을 알고 싶으면 (가)에 `print(result)`를 넣는다.

정답 및 해설

5.2

조건을 충족할 때 반복하기: while

5.2.1 while 문의 형식

반복문에 `for`를 사용하지 않고 다른 방법으로 반복문으로 표현할 수도 있습니다. 바로 `while`입니다. `for` 문에서는 반복할 부분과 반복할 구간을 설정했지만, `while` 문에서는 반복할 부분과 반복할 조건을 설정합니다. 형식은 다음과 같습니다.

형식

```
while 조건:
    명령어
```

`while`이라는 단어는 `for`와 같은 '~동안'이라는 의미가 있습니다. 앞의 형식을 보면 `while` 바로 다음에 조건이 들어갑니다. 이 조건만 충족한다면 `while` 문 내부에 있는 명령어를 반복해서 실행합니다. `for i in range(1, 20)`처럼 자동으로 변수 `i`를 만들고, 값을 차례대로 넣어 주는 구조가 아니기 때문에 `while` 문에서는 조건에 사용될 변수를 미리 만들어 초깃값을 저장하는 과정이 필요합니다. 간단한 예시를 봅시다.

```
num = 0
while num < 10:
    num = int(input('숫자 입력: '))
```

반복문을 실행하기 전에 첫째 줄에서 반복문에 사용할 변수 `num`을 만들고 초깃값을 0으로 저장했습니다. 둘째 줄에서는 `while` 문으로 조건을 충족하는지 확인합니다. 현재 `num`에 저장된 값은 0이므로 `num < 10`을 충족합니다. 따라서 반복문 내부의 `num = int(input('숫자 입력: '))`을 실행해 입력받은 숫자가 `num`에 저장됩니다. 이때 입력받은 숫자가 10 미만이면 셋째 줄의 명령어가 반복되고, 입력받은 숫자가 10 이상이면 반복문을 종료합니다. 프로그램을 실행해 10 미만의 숫자를 몇 개 입력한 후 10 이상의 숫자를 입력해 보세요.

```
실행결과
숫자 입력: 5
숫자 입력: 6
숫자 입력: 12
```

숫자 5, 6에서는 다음 반복이 실행되지만, 12를 입력하면 프로그램이 종료됩니다. 반복 회차별로 일어나는 일을 정리하면 다음과 같습니다.

▼ 표 5-4 반복문의 실행과정

num에 저장되는 값	반복 조건	반복 명령어
0	충족	실행 → 5 입력
5	충족	실행 → 6 입력
6	충족	실행 → 12 입력
12	충족하지 않음	실행되지 않음 → 반복문 종료

만약 첫째 줄의 `num = 0`을 추가하지 않으면 어떻게 될까요? 둘째 줄의 `while` 문을 실행하려면 반드시 그 전에 조건에 사용되는 `num` 변수가 만들어져야 합니다. 그렇지 않으면 다음과 같은 오류가 발생합니다.

```
while num < 10:
    num = int(input('숫자 입력: '))
```

```
실행결과
----> 1 while num < 10:
NameError: name 'num' is not defined
```

`NameError`는 변수가 만들어지지 않아서 발생하는 오류입니다. 만약 오류가 발생하지 않는다면 런타임을 다시 실행하세요. 런타임이란 파이썬 프로그램이 실행(run)되고 있는 시간(time)을 의미합니다. 이전 코드를 실행한 상태, 즉 이전 코드에서부터 런타임이 유지된 상태라면 앞 코드에서 마지막으로 입력한 12가 `num` 변수에 저장된 상태입니다. 따라서 오류가 발생하지 않습니다. 하지만 의도한 대로 명령어가 반복되지 않으니 정확한 프로그램이라 할 수 없습니다.

런타임을 재시작하려면 구글 코랩에서는 [메뉴 → 런타임 → 런타임 초기화]를 클릭합니다. 주피터 노트북에서는 메뉴의 [RUN] 버튼 오른쪽에 있는 돌아가는 화살표 모양 버튼(



restart the kernel)을 클릭합니다. 기타 에디터 프로그램에서도 오류가 발생하지 않는다면 런타임을 초기화하고 재실행해 보세요.

5.2.2 for 문을 while 문으로 바꾸기

앞에서 `for` 문으로 실행했던 프로그램을 `while` 문으로 바꿔 실행해 봅시다.

```
for i in range(1, 20):
    print(i, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
```

변수 `i`의 값이 `range(1, 20)`으로 표현되는 숫자 구간에 있는 동안 출력을 반복하는 예제입니다. 이를 반복 조건으로 바꾸면 `i`가 1부터 20 미만일 때입니다. 다만, `while` 문에서는 반복 조건에 사용할 변수를 미리 만들어야 하므로 `i`에 초깃값을 저장한 후, 1씩 증가된 값을 하나씩 넣어 주어야 합니다.

```
i = 1
while i < 20:
    print(i, '선생님, 다시는 청소를 빼먹지 않겠습니다.')
    i += 1
```

```
실행결과
1 선생님, 다시는 청소를 빼먹지 않겠습니다.
2 선생님, 다시는 청소를 빼먹지 않겠습니다.
...
18 선생님, 다시는 청소를 빼먹지 않겠습니다.
19 선생님, 다시는 청소를 빼먹지 않겠습니다.
```

`i`에 초깃값을 저장하는 코드를 `while` 문 전에 넣어 코드 둘째 줄에서 조건을 검사할 수 있게 합니다. `i`가 `19`일 때까지만 반복하므로 반복 조건을 `i <= 19` 또는 `i < 20`으로 작성합니다(예시에서는 `for` 문과 비교하기 위해 `i < 20`으로 했습니다). 반복문 내부에서 반복할 명령어를 그대로 넣어 줍니다. 그리고 추가로 `i`를 1 증가시키는 `i += 1` 코드가 반드시 필요합니다. `for` 문에서는 반복 구간 안에서 자동으로 다음 데이터를 i에 저장하기 때문에 이런 부분이 필요 없었습니다. 하지만 `while` 문에서는 조건으로 반복 여부를 확인하므로 반복할 때마다 `i`를 1씩 증가시켜 조건 값을 바꿔 줘야 합니다.

NOTE 무한 루프

앞의 코드에서 `i += 1`이 없다면 반복문이 종료되지 않는 문제가 생깁니다. `i = 1`인 상태가 계속 유지되기 때문에 `while` 문의 반복 조건 `i < 20`을 항상 충족하게 됩니다. 그러면 `print(a, '선생님, 다시는 청소를 빼먹지 않겠습니다.')`가 계속 반복되죠. 이를 프로그래밍에서는 무한 루프라고 합니다. 루프(loop)는 '고리, 순환'이라는 의미가 있는데, 무한 루프는 한 마디로 뫼비우스의 띠처럼 프로그램이 무한으로 실행될 뿐 종료되지 않는다는 뜻입니다. 프로그램이 무한 루프에 빠지면 컴퓨터 자원(CPU나 메모리 등)을 계속 차지한 상태가 되어서 다음 코드를 실행하지 못하거나 메모리 부족 등의 문제가 발생합니다.

앞에서 1부터 50까지 숫자의 합을 구하는 프로그램을 `for` 문으로 작성했습니다.

```
result = 0
for i in range(1, 51):
    result += i
print(result)
```

이 프로그램을 `while` 문으로 변경해 봅시다. 직접 변경해 본 후 책을 확인하세요!

```
result = 0
i = 1
while i < 51:
    result += i
    i += 1
print(result)
```

```
실행결과
1275
```

`for` 문은 반복 구간에 초깃값(시작 숫자)과 반복 조건이 들어 있어서 반복 조건 변수를 정의할 필요 없이 반복문 내부에서 `result += i`를 실행하기만 하면 됩니다. 하지만 `while` 문에는 반복 조건만 있기 때문에 변수의 초깃값을 미리 지정하고 변수의 값을 증가시키는 과정이 필요합니다. 따라서 `while` 문이 시작되기 전에 `i = 1`로 변수 `i`를 정의해 반복 구간이 시작하는 값을 별도로 지정해야 합니다. 그러고 나서 `result += i`를 반복할 조건을 `while` 다음에 넣어야 합니다. `i`가 50일 때까지 더하므로 조건은 `i <= 50` 또는 `i < 51`을 넣으면 됩니다. 그리고 `i`를 1씩 증가시키는 `i += 1`을 `while` 문 내부에 반드시 포함해야 합니다. 그렇지 않으면 반복문이 끝나지 않습니다.

반복문을 작성할 때 반복 횟수가 정해져 있거나 반복 구간이 명확하다면 `for` 문을 활용하는 것이 더 편합니다. 그러나 횟수를 지정하지 않고 반복할 조건이 필요할 때는 `while` 문이 더 낫습니다.

5.3

반복문과 조건문 함께 사용하기

5.3.1 조건문을 품은 반복문

이번에는 반복문을 4장에서 배운 조건문과 함께 사용해 보겠습니다.

구구단을 외자(2)

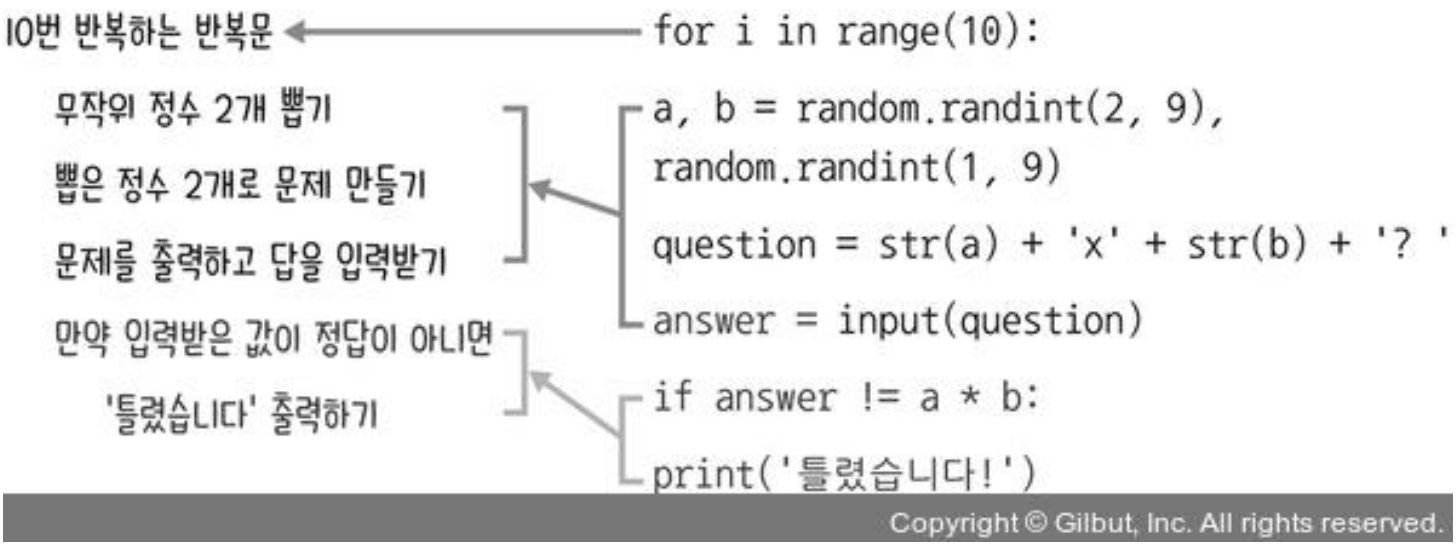
5.1.4 반복문으로 간단한 게임 만들기에서 구구단 게임을 만들었습니다. 그런데 문제가 있었죠. 정답이 아닌 오답을 입력해도 아무런 안내 없이 바로 다음 문제로 넘어갔습니다. 이번에는 입력받은 값이 정답이 아니면 '틀렸습니다!'를 출력해 봅시다. 반복해서 10개 문제를 출제하던 코드에 값이 틀린 경우만 추가하면 됩니다. 어떻게 구현할까요?

앞의 내용을 읽어 보면 '입력받은 값이 정답이 아니면'이라는 조건이 있습니다. 4장에서 배운 조건문으로 표현하면 '만약 입력받은 값이 정답이 아니라면 '틀렸습니다!'를 출력해라'라고 하면 됩니다. 구조를 정리하면 다음과 같습니다.

```
10번 반복하는 반복문
    무작위 정수 2개 뽑기
    두 수를 곱한 값을 물어보고 답을 입력받기
    만약 입력받은 값이 정답이 아니면
        '틀렸습니다!' 출력하기
```

프로그램 흐름을 보니 입력받은 답을 다시 조건문에서 사용합니다. 따라서 입력받은 답을 저장하는 변수가 하나 더 있어야 합니다. 답을 입력받는 코드를 변수 `answer`에 저장합니다. 조건문은 입력받은 값 `answer`와 실제 정답 `a * b`가 다르면(같지 않으면) '틀렸습니다!'를 출력합니다.

기존 코드에 추가된 부분을 작성해 봅시다(코드 부분 들여쓰기 생략) .



코드를 어떻게 작성해야 할지 한눈에 보입니다. 앞에는 표시하지 않았지만, `input()` 명령어로 입력받은 `answer`의 값은 문자열로 인식되므로 정수형으로 변경하는 것도 잊지 말아야 합니다. 그럼 전체 코드를 작성해 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a, b = random.randint(2, 9), random.randint(1, 9)
    question = str(a) + 'x' + str(b) + '? '
    answer = input(question)
    answer = int(answer)
    if answer != a * b:
        print('틀렸습니다!')
```

실행결과

```
구구단을 외자! 구구단을 외자!
9x2? 18
4x4? 4
틀렸습니다!
3x2? 6
8x3? 45
틀렸습니다!
3x5? 15
7x4? 29
틀렸습니다!
2x5? 10
6x8? 46
틀렸습니다!
```

입력에 정답도 넣고 오답도 넣어 보면서 의도하는 대로 결과가 나오는지 확인해 보세요. 오답을 넣으면 '틀렸습니다!'가 출력되고 정답을 입력하면 바로 다음 문제로 넘어갑니다.

코드를 잘 보면 반복문 안에 조건문이 들어 있습니다. 이처럼 필요한 경우 반복문 안에 조건문을 넣어 사용할 수 있습니다. 이런 구조는 실제 프로그램에서 자주 사용합니다.

제제쌤의 조언

정답을 입력했는데도 '틀렸습니다!'가 출력된다면 앞의 코드에서 깜빡하고 `answer`를 `int()`로 감쌌는지 확인하세요. `input()`은 입력값을 모두 문자열로 취급하므로 숫자와 비교하면 항상 다르다는 결과가 나옵니다. 6과 '6'은 컴퓨터가 다르게 인식합니다. 프로그램에서 어려운 부분을 잘 작성해도 데이터 타입을 놓쳐 엉뚱한 결과가 나오는 경우가 많습니다. 항상 데이터 타입에 유의하세요!

NOTE 코드양 줄이기

역시 여기서도 코드를 줄일 수 있습니다.

```
question = str(a) + 'x' + str(b) + '? '
answer = input(question)
answer = int(answer)
if answer != a * b:
```

Copyright © Gilbut, Inc. All rights reserved.

첫째 줄의 `question`과 둘째 줄의 `question`은 같은 값입니다. 따라서 둘째 줄의 `question`을 `str(a) + 'x' + str(b) + '? '`로 대체할 수 있습니다. 그리고 둘째 줄의 `answer`와 셋째 줄의 `answer`도 같은 값입니다. 따라서 셋째 줄도 `answer`도 `input(str(a) + 'x' + str(b) + '? ')`로 바꿔 작성할 수 있습니다.

```
answer = int(input(str(a) + 'x' + str(b) + '? '))
if answer != a * b:
```

Copyright © Gilbut, Inc. All rights reserved.

코드가 네 줄에서 두 줄로 줄죠? 그런데 `answer` 변수를 매개로 한 번 더 합칠 수 있습니다. 역시 첫째 줄의 `answer`와 둘째 줄의 `answer`는 같은 값입니다. 따라서 둘째 줄의 `answer` 대신 `int(input(str(a)+'x'+str(b)+'? '))`를 넣으면 됩니다.

```
if int(input(str(a) + 'x' + str(b) + '? ')) != a * b:
```

두 줄이 한 줄로 줄었습니다. 여기에서도 역시 본인이 원하는 방식에 맞게 변수를 사용해도 되고, 코드양을 줄이고 싶으면 변수를 사용하지 않아도 됩니다.

홀짝 게임

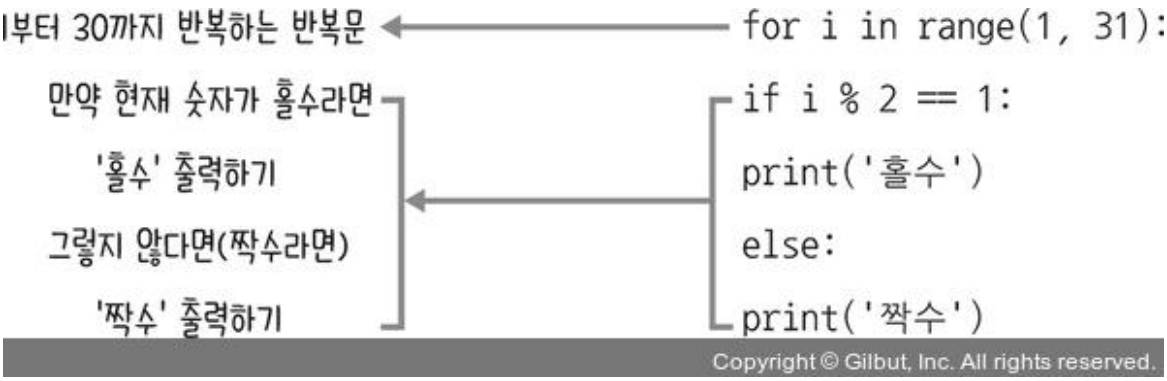
반복문 안에 조건문이 있는 구조를 조금 더 연습해 봅시다. 1부터 30까지 숫자의 홀짝을 출력하는 프로그램을 만들겠습니다. 실행결과는 '1 홀수, 2 짝수, 3 홀수 ... 29 홀수, 30 짝수'입니다.

프로그램을 작성하기 전에 전체 흐름을 한번 생각해 볼까요? 1부터 30까지의 숫자가 차례대로 출력되고, 각 숫자가 홀수인지 짝수인지도 함께 출력됩니다. 일단 반복되는 패턴을 먼저 찾아야 합니다. 숫자와 홀짝이 출력되는 부분이 반복되죠. 반복되는 범위는 1부터 30까지입니다. 따라서 1부터 30까지 반복하는 반복문이 있어야겠네요. 그리고 현재 숫자가 홀수면 '홀수'를 출력하고 그렇지 않으면 '짝수'를 출력해야 하니 홀짝을 판단하는 조건문이 필요하고요. 이를 숫자마다 판단해야 하니 조건문은 반복문 안에 있어야 합니다. 구조를 정리하면 다음과 같습니다.

```
1부터 30까지 반복하는 반복문
    만약 현재 숫자가 홀수라면
        '홀수' 출력하기
    그렇지 않다면(짝수라면)
        '짝수' 출력하기
```

홀수와 짝수를 판단하는 방법은 4장에서 배웠듯이 2로 나눈 나머지를 활용하면 됩니다. 2로 나눈 나머지가 1이면 홀수이므로 '홀수'를 출력하고, 그렇지 않으면 짝수이므로 '짝수'를 출력합니다. 이를 조건문으로 작성합니다.

구조와 코드를 합쳐 봅시다.



코드를 작성하고 결과를 확인해 봅시다.

```
for i in range(1, 31):
    if i % 2 == 1:
        print(i, '홀수')
    else:
        print(i, '짝수')
```

```
실행결과
1 홀수
2 짝수
…
29 홀수
30 짝수
```

4장에서도 배운 내용이니 어렵지 않을 겁니다. 코드가 진행되는 과정을 정리하면 다음과 같습니다.

▼ 표 5-5 반복문의 실행과정

반복 회차	i에 저장되는 값	반복문 내부

첫 번째	1	for i in range(1, 31):
		i에 1 저장
		if i % 2 == 1: 조건 판단: 1을 2로 나눈 나머지는 1이므로 충족
		print(i, '홀수') 'i' 출력 실행
		else:
		print(i, '짝수')

두 번째	2	for i in range(1, 31):
		i에 2 저장
		if i % 2 == 1: 조건 판단: 2를 2로 나눈 나머지는 0이므로 충족하지 않음
		print(i, '홀수')
		else:
		print(i, '짝수') '짝수' 출력 실행

…		

스물아홉 번째	29	for i in range(1, 31):
		i에 29 저장
		if i % 2 == 1: 조건 판단: 29를 2로 나눈 나머지는 1이므로 충족
		print(i, '홀수') '홀수' 출력 실행
		else:
		print(i, '짝수')

서른 번째	30	for i in range(1, 31):
		i에 31 저장
		if i % 2 == 1: 조건 판단: 30을 2로 나눈 나머지는 0이므로 충족하지 않음
		print(i, '홀수')
		else:
		print(i, '짝수') '짝수' 출력 실행

반복문이 실행될 때마다 1부터 30이 차례대로 `i`에 저장되고, 조건문에서 `i`의 값이 조건을 충족하는지 판단합니다. 반복문 안에 조건문이 들어 있으므로 조건문도 30번 반복됩니다.

그런데 결과가 30줄이나 출력되어 한눈에 보기가 불편합니다. 이럴 때 출력 명령어에 `end` 옵션을 추가하면 쉽게 해결됩니다.

```
for i in range(1, 31):
    if i % 2 == 1:
        print(i, '홀수', end=', ')
    else:
        print(i, '짝수', end=', ')
```

```
실행결과
1 홀수, 2 짝수, 3 홀수, 4 짝수, 5 홀수, 6 짝수, 7 홀수, 8 짝수, 9 홀수, 10 짝수, 11 홀수, 12 짝수, 13 홀수, 14 짝수, 15 홀수, 16 짝수, 17 홀수, 18 짝수, 19 홀수, 20 짝수, 21 홀수, 22 짝수, 23 홀수, 24 짝수, 25 홀수, 26 짝수, 27 홀수, 28 짝수, 29 홀수, 30 짝수,
```

`print()`에서 `end`에 원하는 문자를 넣으면 마지막 출력 문자를 지정할 수 있습니다. 앞의 코드처럼 콤마 후 한 칸 공백을 지정하면 줄이 바뀌지 않고 콤마(`,`)를 넣고 한 칸 띄어 쓴 후 다음 결과를 이어서 출력합니다. 콤마 대신에 `'->, /'` 등 원하는 문자를 넣고 테스트해 보세요!

369 게임(1)

이번에는 또 다른 게임인 369 게임을 프로그램으로 만들어 보겠습니다. 369 게임은 1부터 차례대로 숫자를 말하다가 3, 6, 9가 포함된 숫자일 때는 숫자가 아닌 박수를 치는 게임입니다. 예를 들어 13, 26, 32일 때는 숫자를 말하지 않고 박수를 쳐야 합니다.

숫자에 3, 6, 9가 들어 있는지 어떻게 알아낼까요? 이럴 때 사용하는 것이 바로 `in`입니다. `in`은 조건문에서도 사용할 수 있습니다. 다음 코드를 봅시다.

```
if '아버지' in '아버지가방에들어가신다.':
    print('yes')
```

반복문을 처음 배울 때 `in`은 '안에 있다'는 의미라고 했습니다. 이에 맞춰 코드를 해석하면 "'아버지가방에들어가신다.' 안에 '아버지'가 있으면 'yes'를 출력하라"입니다. 굉장히 직관적이죠? 실행하면 'yes'가 출력됩니다.

```
실행결과
yes
```

이번에는 '아버지'가 있으면 'yes'를, 없으면 'no'를 출력하도록 수정해 봅시다.

```
if '아버지' in '아버지가방에들어가신다.':
    print('yes')
else:
    print('no')
```

```
실행결과
yes
```

‘어머니’도 있는지 확인해 볼까요?

```
if '어머니' in '아버지가방에들어가신다.':
    print('yes')
else:
    print('no')
```

```
실행결과
no
```

‘어머니’가 없으므로 ‘no’가 출력되네요. 이때 `in`이 아니라 `not in`으로 작성해도 됩니다. `not in`은 `in`과 반대로 ‘안에 없다’는 의미입니다.

```
if '어머니' not in '아버지가방에들어가신다.':
    print('yes')
else:
    print('no')
```

```
실행결과
yes
```

‘어머니’가 없으면 ‘yes’를 출력하는 코드입니다. `in`과 `not in`은 상당히 직관적이어서 조건문 if에서 자주 사용합니다. 다만, `in`과 `not in`은 문자열에서만 사용할 수 있고, 숫자형에서는 사용할 수 없습니다.

1부터 100까지 숫자를 차례대로 출력하되 3, 6, 9가 들어 있는 숫자에서는 '짝!'을 출력한다고 했을 때 구조를 정리하면 다음과 같습니다.

1 1부터 100까지 반복해야 하니 반복문이 필요합니다.

2 숫자에 3, 6, 9가 있는지 판단해야 해서 반복문 안에 조건문이 필요합니다. 1부터 100까지 숫자가 차례대로 저장되는 `i`에 3 또는 6 또는 9가 있으면 '짝!'을 출력합니다.

정리한 내용으로 코드를 작성해 봅시다. 세 조건(`3 in i`, `6 in i`, `9 in i`) 중 하나만 만족해도 되니 세 조건을 `or`로 연결해 조건문을 작성합니다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    if 3 in i or 6 in i or 9 in i:
        print('짝!', end=' ')
    else:
        print(i, end=' ')
```

```
실행결과
----> 3 if 3 in i or 6 in i or 9 in i:
TypeError: argument of type 'int' is not iterable
```

`TypeError`가 발생합니다. 설명을 보니 조건문에서 오류가 났네요. 앞에서 설명했듯이 `in`에는 문자열만 사용할 수 있는데 `int`, 즉 정수형을 사용해서 생긴 오류입니다.

조건을 `in`에 사용할 수 있는 문자열로 수정해 봅시다. 3, 6, 9 모두 문자로 바꾸고 변수 `i`도 숫자이므로 `str()`로 감싸 문자열로 바꿉시다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    if '3' in str(i) or '6' in str(i) or '9' in str(i):
        print('짝!', end=' ')
    else:
        print(i, end=' ')
```

실행결과

```
1 2 짝! 4 5 짝! 7 8 짝! 10 11 12 짝! 14 15 짝! 17 18 짝! 20 21 22 짝! 24 25 짝! 27 28 짝! 짝!
짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 40 41 42 짝! 44 45 짝! 47 48 짝! 50 51 52 짝! 54 55 짝! 57
58 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 70 71 72 짝! 74 75 짝! 77 78 짝! 80 81 82 짝! 84
85 짝! 87 88 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 짝! 100
```

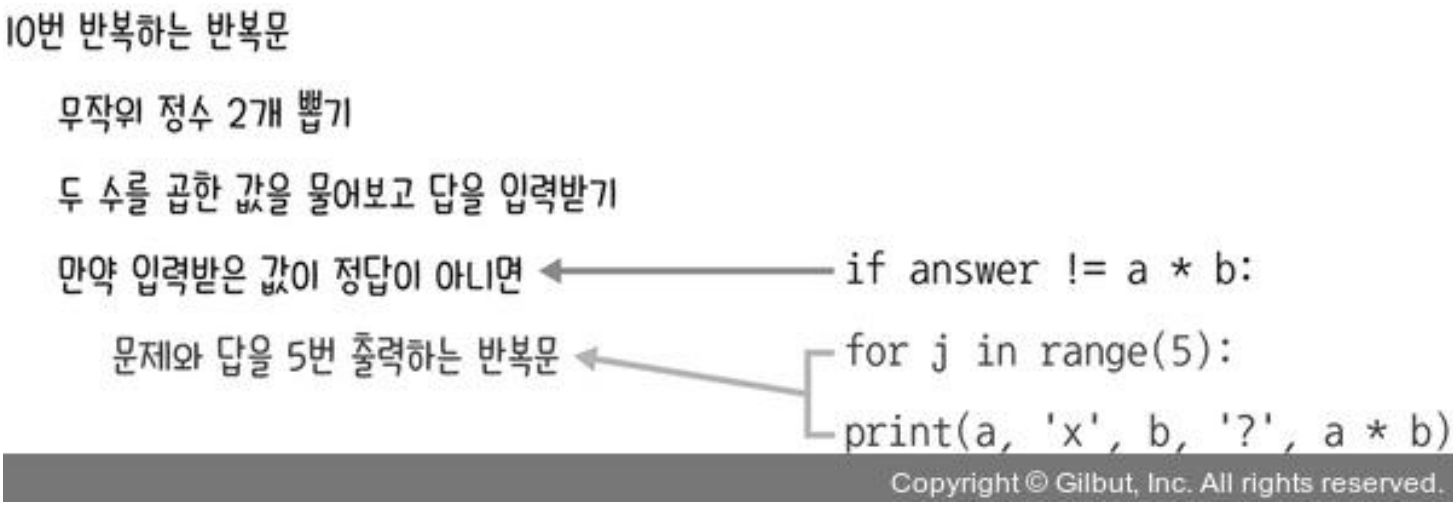
1부터 100까지 오류 없이 잘 출력됩니다. `end` 옵션을 줘서 결과가 줄바꿈되지 않게 했습니다.

결과를 보니 계속해서 박수를 치는 구간이 있습니다. 사람이 369 게임을 하면 중간에 누군가는 틀리기 때문에 100까지 진행되는 경우가 거의 없습니다. 그런데 컴퓨터는 틀린 부분 없이 빠르게 게임을 끝냅니다.

이렇게 출력만 하고 끝나면 너무 시시하니 구구단 게임처럼 답을 입력받아 틀리면 프로그램이 끝나게 해 봅시다. 이 부분은 다음 절에서 다루니 코드를 잠깐 저장해 두었다가 5.4.2 break를 사용한 게임 만들기에서 다시 열겠습니다.

5.3.2 반복문과 조건문 중첩 사용하기

반복문과 조건문은 프로그램 안에서 여러 번 중첩해 사용할 수도 있습니다. 앞에서 작성한 구구단 게임으로 확인해 봅시다. 이번에는 오답을 입력하면 '틀렸습니다! 5번 반복해 읽어 봅시다!'를 출력한 뒤에 정답을 5번 출력하는 프로그램을 작성해 봅시다. 틀렸을 경우 정답을 5번 출력하므로 입력받은 답이 맞는지 확인하는 조건문 안에 반복문이 들어가야 합니다. 구조를 살펴봅시다.



`print(a, 'x', b, '?', a * b)`가 5번 실행되도록 `if` 조건문 내부에 반복문을 작성합니다. 이때 반복문의 변수는 `i`가 아니라 `j`임에 유의하세요! `i`는 이미 상위 반복문(`for i in range(10)`)에서 쓰였기 때문에 같은 변수를 사용하면 컴퓨터가 이해하지 못합니다. 물론 다른 변수를 사용해도 되며 결과에도 영향이 없습니다. 코드를 작성하고 실행해 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a, b = random.randint(2, 9), random.randint(1, 9)
    answer = input(str(a) + 'x' + str(b) + '? ')
    answer = int(answer)
    if answer != a * b:
        print('틀렸습니다! 5번 반복해 읽어 봅시다.')
        for j in range(5):
            print(a, 'x', b, '?', a * b)
```

실행결과

```
구구단을 외자! 구구단을 외자!
6x2? 12
7x6? 42
6x2? 10
틀렸습니다! 5번 반복해 읽어 봅시다.
6 x 2 ? 12
6 x 2 ? 12
6 x 2 ? 12
6 x 2 ? 12
6 x 2 ? 12
4x5? 20
…
```

정답과 오답을 입력해 보고 의도한 대로 결과가 나오는지 확인하세요.

1분 퀴즈

2 코드를 보고 다음 중 틀린 것을 고르세요.

```
for i in range(100):
    if i % 5 == 1:
        print('쿵', end=' ')
    elif i % 5 == 2:
        print('짝', end=' ')
    else:
        print(i, end=' ')
```

① 반복문 내부에 있는 코드를 100번 반복한다.

② 가장 먼저 '쿵'이 출력된다.

③ `i`가 32일 때 '짝'이 출력된다.

④ 마지막으로 99가 출력된다.

⑤ `print()`에 `end` 옵션을 추가했으므로 줄바꿈되지 않고, 한 칸 공백 후 다음 글자가 출력된다.

정답 및 해설

5.4

열심히 달리는 반복문에 브레이크 밟기: break

5.4.1 반복 멈추기

반복문으로 컴퓨터에 일을 시키던 중 실행을 멈추고 싶으면 어떻게 해야 할까요? 예를 들어, 구구단 게임에서 정답을 말하면 그 다음 문제로 진행하지만, 틀렸을 때는 게임을 중단하고 벌칙을 수행하게 하고 싶습니다. 이를 프로그램으로 구현하려면 답이 틀렸을 경우 다음 문제를 내지 않고 반복문을 중단해야 합니다.

달리는 차를 멈출 때 브레이크를 밟듯이 파이썬에서도 반복문을 멈출 때 `break`를 사용합니다. 간단한 코드로 `break`를 어떻게 사용하는지 알아봅시다.

```
for i in range(20):
    print(i)
    break
```

```
실행결과
0
```

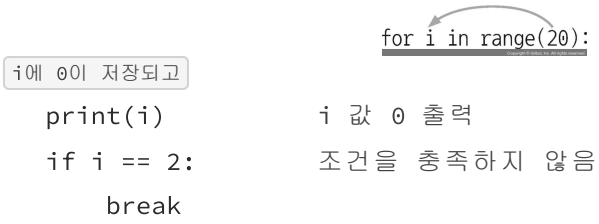
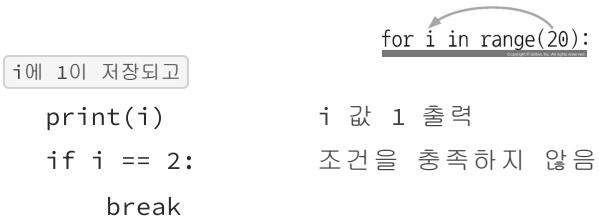
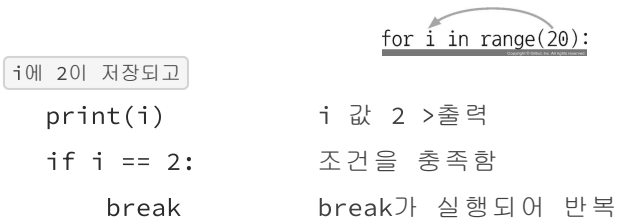
0 하나만 출력한 뒤 프로그램이 종료되네요. 차근차근 살펴봅시다. `range(20)`은 0부터 19(즉, 20-1)까지 진행한다는 의미이므로 변수 `i`에는 0부터 19까지 차례대로 들어갑니다. `i`에 첫 숫자 0이 들어갈 때 `print(i)`가 실행되어 0이 출력됩니다. 그다음에 바로 `break`가 나옵니다. 그러면 `i`에 1이 들어가기 전에 반복문을 종료합니다. 프로그램을 조금 수정해 봅시다.

```
for i in range(20):
    print(i)
    if i == 2:
        break
```

```
실행결과
0
1
2
```

반복문 내부에 있는 조건문은 `i`가 2일 때 반복문을 멈추라는 의미입니다. `i`에 0, 1이 들어가면 조건문을 충족하지 않으므로 `break`가 실행되지 않다가 `i`에 2가 들어가면 조건을 충족해 `break`가 실행되고 반복문을 종료합니다.

▼ 표 5-6 반복문의 실행과정

반복 회차	a에 저장된 값	반복문 내부
첫 번째	0	
두 번째	1	
세 번째	2	

반복문과 조건문의 구조를 잘 파악하면 `break`가 포함된 프로그램도 이해하기 쉽습니다.

이번에는 0부터 19까지의 숫자 중 하나를 입력받으면 해당 숫자까지 출력하고 프로그램을 종료하도록 앞의 프로그램을 수정해 봅시다. 결과가 한 줄로 출력되도록 `end=' '`옵션도 넣습니다.

```
answer = int(input('숫자 입력: '))
for i in range(20):
    print(i, end=' ')
    if i == answer:
        break
```

```
실행결과
숫자 입력: 12
0 1 2 3 4 5 6 7 8 9 10 11 12
```

`input()`으로 숫자를 입력받습니다. 입력받은 숫자를 `int()`로 감싸서 정수형으로 변환하고 변수 `answer`에 저장합니다. 반복문에서 `i` 값을 출력하고 조건문에서 `i`가 `answer` 값과 같은지 비교합니다. 만약 같으면 `break`가 실행되어 반복을 종료합니다.

다음처럼 `print()`와 조건문의 순서를 바꾸면 어떻게 될까요?

```
answer = int(input('숫자 입력: '))
for i in range(20):
    if i == answer:
        break
    print(i, end=' ')
```

```
실행결과
숫자 입력: 12
0 1 2 3 4 5 6 7 8 9 10 11
```

12를 입력하면 이전과 달리 11까지 출력됩니다. 이런 차이가 왜 발생하는 걸까요? 바로 위에서부터 아래로 실행되는 순차구조 때문입니다. 코드를 잘 봅시다. 이전 코드의 반복문에서는 먼저 `print()`로 숫자를 출력한 후에 조건문을 실행합니다. 즉, `i`와 `answer`의 값이 같을 때도 출력 명령어가 실행된 후에 `break`를 만나기 때문에 `i`가 출력되고 나서 반복문이 종료됩니다. 하지만 바뀐 코드에서는 출력 명령어보다 조건문이 더 위에 있기 때문에 `i`와 `answer`의 값이 같으면 `break`가 먼저 실행되어 반복문을 종료합니다. 그래서 입력받은 숫자가 출력되지 않죠. 이처럼 `break`가 어디에 위치하느냐에 따라 결과가 달라지므로 `break`를 사용할 때 유의해야 합니다.

5.4.2 break를 사용한 게임 만들기

`break`의 동작을 이해했으니 앞에서 만든 게임 프로그램을 완성해 보겠습니다.

구구단을 외자(3)

먼저 구구단 게임부터 바꿔 봅시다. 오답을 입력하면 '땡!'을, 정답을 입력하면 정답을 출력한 후 프로그램을 종료하게 수정해 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
for i in range(10):
    a, b = random.randint(2, 9), random.randint(1, 9)
    answer = input(str(a) + 'x' + str(b) + '? ')
    answer = int(answer)
    if answer != a * b:
        print('땡! 정답은', a * b)
        break
```

```
실행결과
구구단을 외자! 구구단을 외자!
4x4? 16
7x9? 74
땡! 정답은 63
```

오답을 입력하면 반복문을 종료하므로 조건문 내부에 출력 명령어와 `break`를 넣으면 됩니다. 오답일 때 프로그램이 종료되니 이제야 진짜 게임 같네요. 정답과 오답을 골고루 입력해서 프로그램이 잘 완성됐는지 확인해 보세요!

`while` 문에서도 `break`를 사용할 수 있습니다. 그런데 `while` 문의 구조를 잘 활용하면 `break`를 사용하지 않고도 반복을 멈출 수 있습니다. `break` 조건을 `while` 문의 조건으로 표현하면 됩니다. `while` 문은 '조건을 충족할 때만 반복을 진행'합니다. 구구단 프로그램은 '반복문 안에서 입력된 숫자가 정답이 아닐 때 반복을 중지'했습니다. 이를 `while` 문으로 바꾸면 '입력된 숫자가 정답일 때만 구구단 반복문을 진행'한다고 할 수 있죠. 그럼 앞의 구구단 프로그램을 `while` 문으로 바꿔 봅시다.

```
import random
print('구구단을 외자! 구구단을 외자!')
answer, a, b = 0, 0, 0
while answer == a * b:
    a, b = random.randint(2, 9), random.randint(1, 9)
    answer = input(str(a) + 'x' + str(b) + '? ')
    answer = int(answer)
print('땡! 정답은', a * b)
```

```
실행결과
구구단을 외자! 구구단을 외자!
7x5? 35
8x4? 32
6x3? 18
8x1? 6
땡! 정답은 8
```

반복 조건은 간단합니다. 입력한 `answer`가 정답 `a * b`와 일치할 때만 구구단 문제를 계속 내면 되므로 `answer == a * b`로 작성하면 됩니다. 그런데 `while`이 등장하기 전에 `while`에서 사용되는 변수가 이미 만들어져야 오류가 발생하지 않았죠? 따라서 `while`이 등장하기 전에 변수 `answer`와 `a`, `b`를 정의해야 합니다. 3개의 변수에 모두 0을 저장하는 코드를 한 줄로 작성합니다.

세 변수에 각각 0을 넣으면 처음 실행할 때도 `answer == a * b`를 충족합니다. 반복문은 정답이 틀리면 종료합니다. 정답이 틀리면 `while` 문을 종료하고 다음 코드로 넘어가므로 정답이 틀릴 때 출력하는 명령어는 `while` 문 외부에 마지막으로 적어주면 됩니다.

369 게임(2)

이번에는 앞에서 간단하게 작성한 369 게임을 발전시켜 보겠습니다. 1부터 100까지 숫자를 출력하되 3, 6, 9가 포함된 숫자에서는 '짝!'을 출력하도록 만든 프로그램이었습니다. 사실 컴퓨터가 혼자 하는 프로그램이죠. 이번에는 게임처럼 입력칸을 만들어 사람이 직접 숫자를 입력하도록 만들겠습니다.

순서에 맞는 숫자나 '짝'이 입력되면 다음 입력으로 넘어가고, 잘못된 입력이 들어오면 '틀렸습니다!'를 출력한 후 프로그램을 종료합니다. 먼저 입력받는 부분을 추가해 봅시다. 100이 될 때까지 숫자를 계속 입력받아야 하므로 반복문 안에 `input()`을 넣고, 입력된 값은 변수 `say`에 저장합니다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    say = input()
```

다음으로 입력값이 올바른지 확인해야죠. 숫자에 3, 6, 9가 들어 있을 때(`if`)부터 봅시다. 3, 6, 9가 들어 있는 숫자에서는 `say`에 저장된 값이 '짝'이어야 합니다. '짝'이 아니면 '틀렸습니다!'를 출력하고 바로 프로그램을 종료하면 됩니다. 따라서 첫 번째 조건문 내부에 `say` 값이 짝이 아닌지를 판단하는 새로운 조건문을 넣습니다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    say = input()
    if '3' in str(i) or '6' in str(i) or '9' in str(i):
        if say != '짝':
            print('틀렸습니다!')
            break
```

이번에는 숫자에 3, 6, 9가 없는 경우(`else`)를 수정해 보겠습니다. 현재 숫자는 `i`에 저장되어 있습니다. 따라서 `i`와 `say`에 저장된 입력값이 같지 않으면 '틀렸습니다!'를 출력하고 `break`로 종료하면 됩니다. 입력값은 문자열로 인식하므로 `say`의 데이터 타입은 문자열입니다. 따라서 `i`를 `str()`로 감싸서 문자열로 바꿔야 `say`와 비교할 수 있습니다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    say = input()
    if '3' in str(i) or '6' in str(i) or '9' in str(i):
        if say != '짝':
            print('틀렸습니다!')
            break
    else:
        if say != str(i):
            print('틀렸습니다!')
            break
```

프로그램을 실행해 보세요! '짝'을 입력해야 하는데 숫자를 입력한 경우, 틀린 숫자를 입력한 경우 등 다양하게 넣으며 `break`가 제대로 동작하는지도 확인해 보세요.

> 실행결과
> '짝'을 입력해야 하는데 숫자를 입력한 경우
> 3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~
> 1
> 2
> 3
> 틀렸습니다!
>
> 숫자를 입력해야 하는데 '짝'을 입력한 경우
> 3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~
> 1
> 2
> 짝
> 짝
> 틀렸습니다!
>
> 틀린 숫자를 입력한 경우
> 3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~
> 1
> 2
> 짝
> 5
> 틀렸습니다!

NOTE say 변수를 숫자형으로 바꾼다면?

say가 문자열이고 i가 숫자형이기 때문에 앞에서는 i를 문자열로 변경했습니다. 이때 say를 숫자형으로 바꾸면 어떻게 될까요? 일부 입력에서 오류가 발생합니다. 예를 들어, 숫자를 입력해야 하는데 '짝'을 입력하는 경우입니다. 코드를 변경하고 실행해 봅시다.

```
print('3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~ ')
for i in range(1, 101):
    say = input()
    if '3' in str(i) or '6' in str(i) or '9' in str(i):
        if say != '짝':
            print('틀렸습니다!')
            break
    else:
        if int(say) != i:
            print('틀렸습니다!')
            break
```

실행결과

```
2 대신 '짝'을 입력한 경우
3~ 6~ 9~ 369~ 3~ 6~ 9~ 369~
1
짝
---------------------------------------------------------------------------
---> 9         if int(say) != i:
ValueError: invalid literal for int() with base 10: '짝'
```

숫자 입력 순서에 '짝'을 입력하니 오류가 납니다. ValueError는 값과 관련된 오류로, 3장에서 접한 적이 있습니다. 오류 내용을 해석해 보니 "int()에 유효하지 않은 글자"라고 합니다. 이는 문자열인 '짝'을 저장한 say를 int()로 감싸도 정수형으로 바꿀 수 없어서 생긴 오류입니다. 하지만 숫자가 저장된 i는 문자열로 바꿀 수 있으므로 say != str(i)로 비교하는 것이 옳은 방법입니다.

1분 퀴즈

3 다음 코드를 보고 옳지 않은 것을 고르세요.

```
for i in range(20):
    (가)
    if i % 2 == 0:
        print('짝수')
        (나)
    else:
        (다)
        print('홀수')
        (라)
    (마)
```

① (가)에 `break`를 넣으면 아무것도 출력되지 않고 프로그램이 종료된다.

② (나)에 `break`를 넣으면 '짝수'가 한 번 출력되고 프로그램이 종료된다.

③ (다)에 `break`를 넣으면 '짝수'가 한 번 출력되고 프로그램이 종료된다.

④ (라)에 `break`를 넣으면 '홀수'가 한 번 출력되고 프로그램이 종료된다.

⑤ (마)에 `break`를 넣으면 '짝수'가 한 번 출력되고 프로그램이 종료된다.

정답 및 해설

5

마무리

이 장에서 배운 내용을 정리해 보겠습니다.

1 제어구조의 종류

종류	진행 순서
순차구조	위에서 아래의 순서로 실행되는 구조
선택구조(조건문)	조건 충족 여부에 따라 실행할 명령어가 달라지는 구조
반복구조(반복문)	특정 명령어를 지정된 범위 또는 조건 내에서 반복하는 구조

2 반복문의 실행 원리

형식	의미	예시
`for a in data:` `B`	a가 data 구간 안에 있는 값일 동안 B를 실행하라	`for a in [1, 2, 3, 4, 5]:` `  print('안녕하세요')`

`while 조건:`
`B`
	조건을 충족하는 동안
	B를 실행하라
		`a = 1`
		`while a <= 5:`
		`  print('안녕하세요')`
		`  a += 1`

3 연속된 숫자 범위를 나타내는 range()

구분	설명
`range(시작 숫자, 종료 숫자)`	시작 숫자부터 (종료 숫자 - 1) 사이의 연속된 숫자(1씩 증가)
`range(종료 숫자)`	0부터 (종료 숫자 - 1) 사이의 연속된 숫자(1씩 증가)
`range(시작 숫자, 종료 숫자, 간격)`	시작 숫자부터 (종료 숫자 - 1)까지 간격만큼 증가

4 라이브러리 사용하기

구분	설명
라이브러리	특정 기능과 관련된 명령어를 한 데 묶어 놓은 명령어 모음
라이브러리를 포함하는 방법	`import 라이브러리명` 예 `import random`
라이브러리에 속한 명령어 사용법	`라이브러리명.명령어()`

5 random 라이브러리의 명령어

구분	설명
`random.random()`	0부터 1 사이의 무작위 실수
`random.randrange(시작 숫자, 종료 숫자)`	시작 숫자부터 (종료 숫자 - 1) 사이의 무작위 정수
`random.randrange(시작 숫자, 종료 숫자, 간격)`	시작 숫자부터 (종료 숫자 - 1) 사이에서 간격만큼 떨어진 정수 중 무작위
`random.randint(시작 숫자, 종료 숫자)`	시작 숫자부터 종료 숫자(종료 숫자 포함) 사이의 무작위 정수

6 break 반복문을 멈추는 명령어로, `break`가 실행될 조건을 조건문으로 만들 수 있다.

```
for i in data: # 반복문을 실행하다가
    if 조건:   # 조건이 충족되면
        break  # 반복문을 멈춘다
```

Self Check

1 10부터 100까지의 숫자를 10 간격으로 출력합니다. 주어진 반복문을 사용해 프로그램을 완성하세요.

```
for i in range(1, 11):
    (가)
```

2 다음은 홈트레이닝 계획을 출력하는 코드와 그 실행결과입니다. 실행결과를 참고해 (가)와 (나)에 들어갈 조건문을 완성하세요.

```
print('홈트레이닝 2주 계획입니다.')
for i in range(1, 15):
    (가)
        print(i, '일: 휴식일입니다.')
    (나)
        print(i, '일: 스쿼트 - 윗몸 일으키기 - 유산소 운동')
```

실행결과

```
홈트레이닝 2주 계획입니다.
1 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
2 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
3 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
4 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
5 일: 휴식일입니다.
6 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
7 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
8 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
9 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
10 일: 휴식일입니다.
11 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
12 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
13 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
14 일: 스쿼트 - 윗몸 일으키기 - 유산소 운동
```

힌트 휴식하는 날의 특징을 잘 생각해 보세요. 휴식일일 경우와 그렇지 않은 경우로 나눠 생각하면 됩니다.

3 이름과 횟수를 입력받아, 입력받은 횟수만큼 이름을 말하는 게임 프로그램을 작성하려고 합니다. 게임은 5번만 실행된다고 가정합니다. 다음 빈칸에 알맞은 명령어를 넣어 실행결과처럼 나오도록 프로그램을 완성하세요.

```
print('팅 팅팅팅 탱 탱탱탱 팅팅 탱탱 후라이팬 놀이!')
for i in range(5):
    이름과 숫자를 입력받아 입력된 숫자만큼 이름을 반복 출력하는 코드 작성
```

실행결과

```
# 수진-3, 명수-2, 현지-4, 동식-3, 지연-1 입력 시
팅 팅팅팅 탱 탱탱탱 팅팅 탱탱 후라이팬 놀이!
이름? 수진
횟수? 3
수진수진수진
이름? 명수
횟수? 2
명수명수
이름? 현지
횟수? 4
현지현지현지현지
이름? 동식
횟수? 3
동식동식동식
이름? 지연
횟수? 1
지연
```

힌트 반복되는 부분과 반복할 구간을 먼저 찾으세요. 이름과 횟수를 입력받으므로 각각 다른 변수에 저장한 후 입력된 횟수만큼 이름을 출력하면 되겠죠? 문자를 숫자만큼 반복해서 출력하는 방법은 앞에서 배웠습니다. '별표 `*` 100'을 생각해 보세요.

정답 및 해설